Explaining Human-perceived Difficulty of Chess Positions: Survey and Preliminary Experiments

Zala Urh
zu0476@student.uni-lj.si
Faculty of Computer and Information Science, University of Ljubljana
Večna pot 113
SI-1000 Ljubljana, Slovenia

Matej Belšak
mb4390@student.uni-lj.si
Faculty of Computer and Information Science, University of Ljubljana
Večna pot 113
SI-1000 Ljubljana, Slovenia

Luka Bajić
luka.bajic@fri.uni-lj.si
Faculty of Computer and Information Science, University of Ljubljana
Večna pot 113
SI-1000 Ljubljana, Slovenia

Ivan Bratko
ivan.bratko@fri.uni-lj.si
Faculty of Computer and Information Science, University of Ljubljana
Večna pot 113
SI-1000 Ljubljana, Slovenia

ABSTRACT

In this paper we survey existing approaches for estimating difficulty of chess positions as perceived by human players. Additionally, we offer our own implementation of rating prediction on chess puzzles using hand-crafted features, primarily based on the properties of the search tree and on performance of weak engines. We explain both classifiers and regressors using SHAP and LIME.

KEYWORDS

chess, explainability, classification, regression, feature engineering

1 INTRODUCTION

Explaining why a given chess position is perceived as easy or hard has been a long-standing goal both for human players and artificial intelligence researchers. This can be seen as a subset of the problem of explaining game states in general zero-sum two-player games. Possible applications of a solution to this problem range from tutoring software to automatic skill estimation in said games.

In this paper we overview existing approaches in this domain, as well as offer our own implementation of puzzle difficulty prediction. This problem could be formulated in many different ways, and could easily be seen as an interdisciplinary endeavor, however in this work we take a primarily machine learning view. In particular, we think of difficulty estimation as either a regression problem, where given an arbitrary chess position, we simply wish to predict its rating, or a a classification problem, where we should first bin all available positions into classes, say, *easy*, *medium*, and *hard*.

A possible issue with such approaches is the apparent subjectivity of rating assignment, however it will become clear that the process is not unlike assigning ratings to human players and adjusting them based on tournament, or individual game performances, which seems to be a generally accepted method of estimating players' skill level.

Another possible approach would be to avoid numerical ratings altogether, and instead treat difficulty estimation as purely a ranking problem. In this paradigm, the model should simply state which of the given two (or more) positions is perceived as more difficult. However, it is clear that this does not eliminate subjectivity whatsoever.

2 RELATED WORK

In recent years we have observed an increased interest in the problem of predicting puzzle difficulty, largely due to the IEEE BigData 2024 Cup [42] and the FedCSIS 2025 knowledgepit.ai Competition [41], both of which focused precisely on this topic.

The best performing models in both cases belonged either to the deep learning paradigm, e.g. [22, 28], or to the gradient boosting family, e.g. [3]. Interestingly, even the former approaches often utilized hand-crafted features, which is seldom seen in other fields, however it is a good indication of the difficulty of this problem.

A number of hand-crafted features are also described in [6, 7, 35]. Here the idea is to use the breadth and depth of the search tree explored by the engine as a proxy for the difficulty of the given position. For this approach to work, the pruning should coincide with the human thinking process, e.g. checks, and captures should be considered no matter what, while certain nonsensical moves can be rejected instantly without further search.

2.1 Human-play Modelling

Since chess engines started surpassing the skill of human players by a large margin, attention has somewhat shifted to developing engines that exhibit a more human playing style with the purpose of making the moves more intuitively understantable. This is by no means a solved problem, as it is difficult to even quantify what constitutes a human-like playing style.

Arguably the first big breakthrough in this regard was the AlphaZero engine [34], which was trained purely via self-play, and was claimed to produce more human-like moves compared to other state-of-the-art engines at the time. This claim, as well as the possible drawbacks, were discussed, for instance, in [5].

Other noteworthy human-modelling engines come from the Maia family, namely Maia-1 [21], Maia-2 [37], and Maia-3 [23]. The latter, in particular, simultaneously models human play, maximizes playing strength and offers interpretability.

ALLIE [40] is yet another recent attempt at modelling human play, with particular emphasis on predicting the next move based on the history of preceding moves. Similar approach can be seen in Otter [31], but with additional capability of modelling clock management.

2.2 Machine Learning for Tabular Data

Recently, substantial progress has been made in the field of machine learning for tabular data, which can be seen for instance on the TabArena benchmark [12]. Recent foundational models such as Kumo-Tabular [24] and TabFM [19] have surpassed previous state-of-the-art models such as XGBoost [8], CatBoost [10], LightGBM [18], and their variants, all based on the gradient boosting paradigm. Importantly, these models work out-of-the-box, i.e. require no additional domain-specific fine-tuning.

2.3 Engine Strength Estimation

The problem of assigning a rating to chess engines has been of interest even before their level of play was comparable to that of a human [2, 39]. The initial belief was that an estimated Elo rating grows linearly with the depth of engine's search capabilities, i. e. that adding a ply of depth increases the engine's Elo by a constant. Due to hardware limitations, diminishing returns of increasing search depth were only observed later, e.g. in [17].

Currently, engines are evaluated according to the Elo rating system, with rating changing based on tournament play, e.g. [9, 16, 32]. However, engine tournaments are not universally standardized, and can use different time controls, hardware, etc. which means that ratings from one engine to another are not necessarily comparable. They are certainly not directly comparable with human ratings either, despite using the same rating system.

2.4 Explainability

Chess has proven to be a relevant and relatively challenging domain for Explainable AI (XAI). A lengthy survey of recent approaches can be found in [4], for a more general XAI survey, see [33].

State-of-the-art machine learning models, including those discussed in Section 2.2, typically do not offer any inherent explanation for their decision. This is true both for classifiers and regressors, and is especially problematic in the deep learning paradigm, where even the features are usually computed by the model itself, i.e. without human input. The lack of human-understandable features makes it difficult to infer the reasoning behind the output produced by the model. Nonetheless, several approaches to alleviate this problem have been developed.

SHAP [20] is a model-agnostic explainer, meaning the explanation is not tied to the specifics of a given classifier or regressor. This is beneficial because we can thus use the same method to evaluate vastly different models. A high level overview of SHAP is the following: to determine the importance of each individual feature, we can iteratively corrupt different subsets of features (e.g. by setting them to the corresponding mean values) and observe the change in the output of the model. If the change is substantial, the corrupted features were important. Since there are exponentially many subsets of features, Monte Carlo techniques are often used for efficiency. Details of this approximation are discussed in [43].

LIME [30] is a model-agnostic explainer that uses input perturbation to train a surrogate model to locally explain individual predictions.

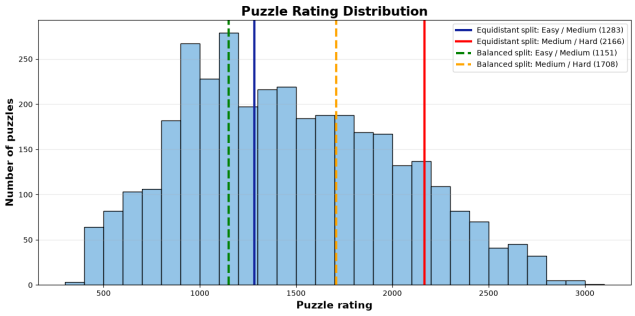


Figure 1: Rating distribution with indicated class delimiters for equidistant split (according to rating) and balanced split (according to number of samples in each class).

3 METHODOLOGY

3.1 Data

To train and evaluate our models, we use a subset of the Lichess puzzle dataset [1] containing 3500 puzzles with ratings ranging between 399 and 3050. We discard all columns except FEN, rating, and themes. Furthermore, we only keep a subset of themes which have a precise definition (for a full list, see the implementation [2]). For classification we create classes "easy", "medium" and "hard" by splitting this interval equidistantly. The dataset is split into training set and test set with the ratio 80:20. The data is standardized, meaning we transform it in such a way that it follows a standard normal distribution with zero mean and unit variance. This is required by distance-based models to ensure that each attribute is treated as equally important.

The distribution with indicated bins for easy, medium, and hard classes, is shown in Figure 1. Classes can be constructed in two sensible ways - one by creating rating bins equidistantly, one by ensuring each bin contains the same number of samples. The latter produces a significantly narrower rating range for *middle* difficulty, which is due to the relatively low frequency of difficult puzzles in the dataset.

All experiments in Chapter 4 use balanced split, where the boundaries are determined on the train set only.

3.2 Evaluation Metrics

For classification problem we use the standard metrics: classification accuracy, F1 score and AUC. For regression we use mean absolute error (MAE) because unlike mean squared error, it has a

[1] https://database.lichess.org/#puzzles

[2] Full implementation is available at https://github.com/bajicluka01/chess-puzzle-evaluation/tree/main

more intuitive interpretation on this dataset. We also report which percentage of the rating interval this MAE covers.

3.3 Features

In this section we discuss the features that were used for model training in Chapter 4.

3.3.1 Meaningful Search Tree. We use the features from [35] (except *SeeminglyGood*, which is not rigorously defined and cannot be easily reproduced) with the following modification: we generate the *MeaningfulMoves* feature using Otter [31], a recently developed engine that models human play, and use these moves to compute the remaining features from [35].

This change is motivated by the following: meaningful moves in [35] are intended to represent the moves a human player would generally consider in a given position (including moves that are quickly rejected, e.g. captures that obviously do not work). However, these moves are computed directly from the engine evaluation, which renders the feature useless in many cases, e.g. in tactical positions in which there is objectively only one good move, whereas a human player clearly needs to consider multiple alternatives. Conversely, Otter [31] is a transformer-based approach that for a given position P returns a probability distribution across all legal moves in P, indicating how likely a human player is to consider each alternative. In other words, we obtain probabilities p_i with the following properties:

$$\forall i = 1, \ldots, k: \quad p_i \geq 0, \quad \sum_{L(P)} p_i = 1 \tag{1}$$

where $L(P)$ is the set of legal moves in position P and i enumerates all the moves in $L(P)$, so $|L(P)| = k$.

In our experiments we select meaningful moves by using an arbitrarily chosen cutoff $p_i > 0.1$, which based on Equation (1) gives at most 10 (but typically less) moves most likely to be considered by a human player in a given position. In our view this coincides with the number of moves an expert player typically considers.

3.3.2 Weak Engines. The following reasoning should make intuitive sense: if a weak engine (or player) can solve a given puzzle, it is more likely to be a low-rated puzzle. Therefore we can employ several weak engines and use indicators of their puzzle-solving success as a subset of features. What exactly is meant by "weak" is not of great importance (see discussion in Chapter 4.1) - in fact it is desirable to use multiple engines with varying estimated strengths [3] to obtain less correlated features.

3.3.3 Miscellaneous. In addition to the above mentioned features, we also use Stockfish's outputs, namely centipawn evaluations, win/draw/loss rate and selective depth.

Furthermore, we compute the number of pins and forks in the given position based on the following definitions.

[3]Note that using, e.g. Stockfish with different playing strength parameters, gives unsatisfactory results. Evidently, making a strong engine play at less-than-full strength, is a harder (and perhaps less interesting) problem than implementing a weak engine from scratch.

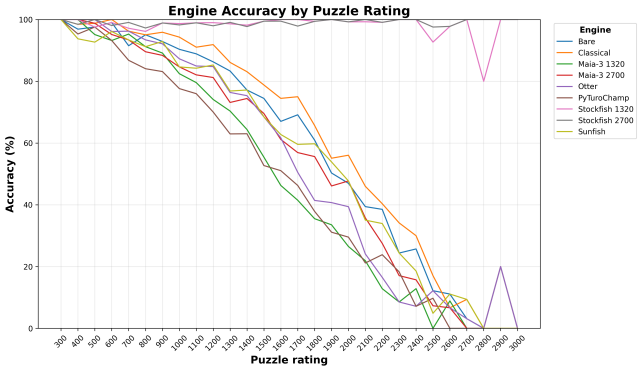


Figure 2: Engine puzzle solving success based on puzzle rating.

4 RESULTS

4.1 Correlation between puzzle rating and engine Elo

The quality of the Elo [1, 11] rating system in chess has been called into question numerous times, e.g. in [15, 26], however it remains a generally accepted measure of playing strength. In many settings, including Lichess (hence also in the dataset used in this paper), the Glicko-2 [13, 14] rating system is used instead [4]. On the other hand, estimation of chess engines' playing strength is generally still done using Elo. Therefore it is worth investigating the correlation between an engine's rating and its ability to solve puzzles of a similar rating.

We use the following engines: Stockfish [38] with two strength settings, 1320 and 2700, Sunfish [36], Otter [31], Classical [29], PyTuroChamp [27] and its base mode, and Maia-3 [23] with same two strength settings as Stockfish. The results are shown in Figure 2. We can observe that all of these engines except Stockfish follow the expected downward trend for increasing puzzle difficulty, therefore the puzzle solving success indicator for all except Stockfish can be used as features.

4.2 Classification

In Table 1 we show the results for the classification problem using balanced class split. Logistic regression from [25] and XGBoost [8] are used as a baseline, and are clearly outperformed by the state-of-the-art models KumoTabular [24] and TabFM [19]. Both of these are used out-of-the-box, without domain-specific fine-tuning. Ensemble refers to a soft voting scheme applied to the outputs of all other models, meaning that probabilistic predictions are aggregated and converted into a class value.

4.2.1 Misclassifications. Due to the ordinal nature of the classes and the somewhat fluid boundaries between them, it makes sense to take a closer look at misclassifications, which we display in Figure 3. As expected, strong errors (i.e. classifying an easy example as hard,

[4]Whether applying any rating system to an individual position rather than a player/engine even makes sense is a separate discussion, which we do not engage in here.

Table 1: Classification scores on 3.5k dataset.

model	CA	F1	AUC
LogReg			
XGBoost [8]	0.653	0.653	0.847
Kumo-Tabular [24]	**0.700**	**0.700**	0.870
TabFM [19]	0.700	0.700	0.865
Ensemble	0.615	0.612	**0.779**

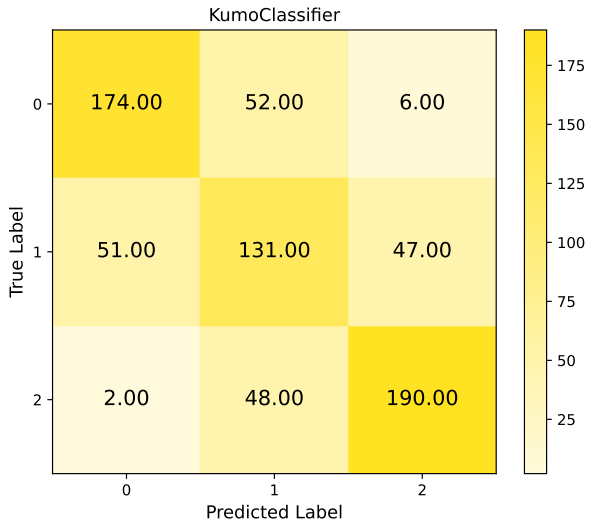


Figure 3: Confusion matrix for KumoTabular classifier.

or vice versa) and significantly less frequent than weak errors (i.e. easy to medium, medium to hard, or vice versa).

4.2.2 Ablation. In Table 2 we show the results of an ablation study aimed to determine the utility of each subset of features. We use the best performing model in terms of CA from the previous experiment, namely KumoTabular. Baseline refers to simple attributes, such as centipawn evaluations and number of searched nodes for each depth. Other entries in the table denote which subsets of features from Chapter 3.3 are used in addition to the baseline.

4.3 Regression

In Table 3 we show the results for the regression problem. Linear regression from [25] and XGBoost [8] are used as a baseline, and are again outperformed by KumoTabular [24] and TabFM [19] without fine-tuning. Ensemble model performs unweighted averaging on the outputs of all other models.

4.4 Explanations

In this section we discuss LIME and SHAP explanation of an arbitrarily chosen position shown in Figure 4. In both cases we explain the KumoTabular regression model.

Table 2: Ablation study on 3.5k dataset.

model	CA	F1	AUC
baseline	0.64	0.62	0.76
SearchTree	0.69	0.67	0.81
Engines	0.71	0.71	0.85
themes	0.66	065	0.79
Misc	0.67	0.65	0.80
SearchTree, Engines	0.73	0.73	0.86
SearchTree, themes	0.70	0.68	0.83
SearchTree, Misc	0.69	0.67	0.82
Engines, themes	0.73	0.73	0.86
Engines, Misc	0.74	0.74	0.87
themes, Misc	0.68	0.67	0.82
all features	0.74	0.74	0.87

Table 3: Regression scores on 3.5k dataset.

model	MAE	% of interval
XGBoost [8]	255.921	12.2
Linear regression [25]	250.721	11.9
Kumo-Tabular [24]	229.471	11.8
TabFM [19]	255.921	11.7
Ensemble	**310.385**	**11.5**

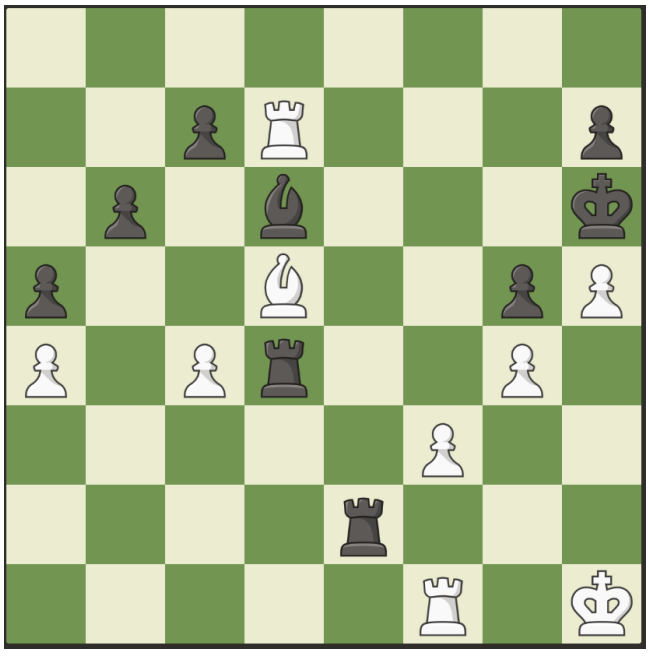


Figure 4: Chess position.

4.4.1 LIME. Figure 5 contains the LIME explanation of the position from Figure 4. 20 most important features are visualized. The y-axis shows the feature values, the x-axis shows the amount of rating points added or subtracted from the mean for each of the features.

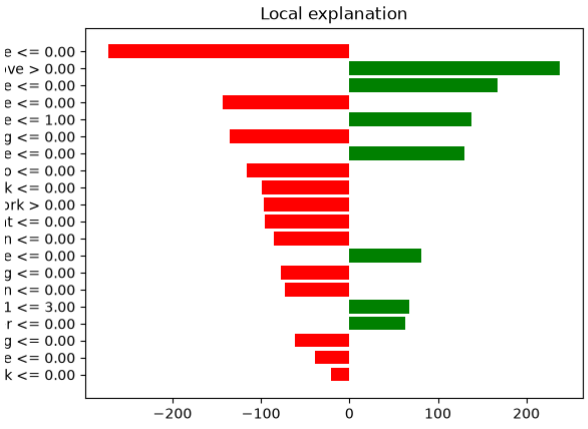


Figure 5: LIME explanation of KumoTabular regressor on the selected chess position. 20 most significant features are shown.

4.4.2 SHAP. In Figure 6 we have the SHAP explanation of the position from Figure 4. Features are again ranked based on importance.

5 CONCLUSIONS

We have surveyed the existing approaches for predicting chess puzzle difficulty, with particular emphasis on hand-crafted features that provide meaningful explanations of the underlying machine learning models. We also provided our own implementation of additional features, including puzzle solving success of weak engines.

The results on classification and regression tasks show that state-of-the-art tabular data models KumoTabular [24] and TabFM [19] outperform the baseline models without any additional fine-tuning. Ablation study demonstrates the utility of weak engine features, although their meaningfulness in explainers is limited.

Future work would consist of checking if these approaches are suitable also for strategic/nondescript positions, rather than just puzzles. Here, the main challenge is in assigning the ground truth ELO to these positions, since consensus among human players is more difficult to achieve. One possible approach would be to take as difficult examples the mistakes from grandmaster games, excluding those made under time pressure. A further limitation is that none of the engine attributes are applicable because there is no ground truth solution.

Additionally, ranking approaches could also be tested. In this setting there is no need for precise rating assignment, which is a convenient property for strategic positions. However, the absence of ground truth still remains an unresolved problem.

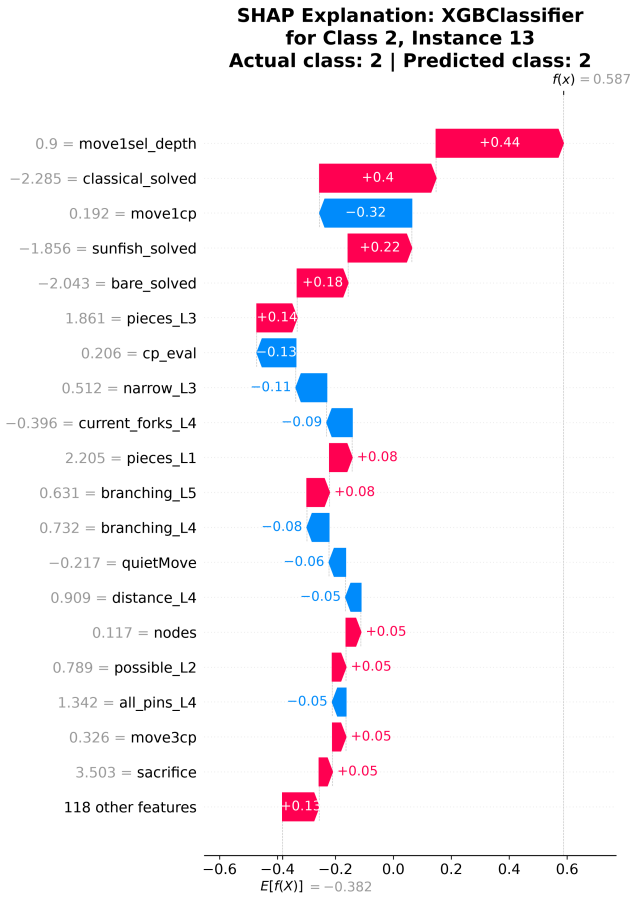


Figure 6: SHAP explanation of KumoTabular r egressor on the selected chess position. Features are ordered by importance.

REFERENCES

[1] Elo Arpad. 1978. The rating of chessplayers, past and present. *Arco Pub* 216 (1978).

[2] Hans J. Berliner, Gordon Goetsch, Murray S. Campbell, and Carl Ebeling. 1990. Measuring the performance potential of chess programs. *Artificial Intelligence* 43, 1 (1990), 7–20. doi:10.1016/0004-3702(90)90067-A

[3] Sebastian Björkqvist. 2024. Estimating the Puzzlingness of Chess Puzzles. In *2024 IEEE International Conference on Big Data (BigData)*. 8370–8376. doi:10.1109/BigData62323.2024.10825991

[4] Yngvi Björnsson. 2024. Chess and explainable AI. *ICGA Journal* 46, 2 (2024), 67–75. arXiv:https://journals.sagepub.com/doi/pdf/10.3233/ICG-240256 doi:10.3233/ICG-240256

[5] Ivan Bratko. 2018. AlphaZero – What's missing? *Informatica (Slovenia)* 42 (01 2018), 7–11.

[6] Ivan Bratko, Dayana Hristova, and Matej Guid. 2016. *Search Versus Knowledge in Human Problem Solving: A Case Study in Chess.* Vol. 27. 569–583. doi:10.1007/978-3-319-38983-7_32

[7] Ivan Bratko, Dayana Hristova, and Matej Guid. 2021. Predicting Problem Difficulty in Chess. In *Human-Like Machine Intelligence*, Stephen Muggleton and Nicholas Chater (Eds.). Oxford University Press. arXiv:https://academic.oup.com/book/0/chapter/350718010/chapter-pdf/43431216/oso-9780198862536-chapter-24.pdf doi:10.1093/oso/9780198862536.003.0024

[8] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*. ACM, 785–794. doi:10.1145/2939672.2939785

[9] Paul Dailly, Dominik Gotojuch, Neil Henning, Keir Lawson, Alec Macdonald, Tamerlan Tajaddinov, and Lilybank Gardens. 2008. A Chess Engine. *University of Glasgow Department of Computing Science Sir Alwyn Williams Building Lilybank Gardens Glasgow G12 8QQMarch* 18 (2008).

[10] Anna Veronika Dorogush, Vasily Ershov, and Andrey Gulin. 2018. CatBoost: gradient boosting with categorical features support. arXiv:1810.11363 [cs.LG] https://arxiv.org/abs/1810.11363

[11] Arpad E. Elo. 1967. The proposed USCF rating system, its development, theory, and applications. *Chess Life* 22, 8 (1967), 242–247.

[12] Nick Erickson, Lennart Purucker, Andrej Tschalzev, David Holzmüller, Prateek Mutalik Desai, David Salinas, and Frank Hutter. 2025. TabArena: A Living Benchmark for Machine Learning on Tabular Data. arXiv:2506.16791 [cs.LG] https://arxiv.org/abs/2506.16791

[13] Mark E. Glickman. 1999. Parameter Estimation in Large Dynamic Paired Comparison Experiments. *Journal of the Royal Statistical Society Series C: Applied Statistics* 48, 3 (09 1999), 377–394. doi:10.1111/1467-9876.00159

[14] Mark E. Glickman. 2012. Example of the Glicko-2 system. http://www.glicko.net/glicko/glicko2.pdf Available at http://www.glicko.net/glicko/glicko2.pdf.

[15] Mark E Glickman and Albyn C Jones. 1999. Rating the chess rating system. *Chance-Berlin Then New York*- 12 (1999), 21–28.

[16] Guy Haworth and Nelson Hernandez. 2020. The 19th Top Chess Engine Championship, TCEC19. *ICGA Journal* 42, 4 (2020), 295–305. arXiv:https://journals.sagepub.com/doi/pdf/10.3233/ICG-200172 doi:10.3233/ICG-200172

[17] Ernst Heinz. 2001. *New Self-Play Results in Computer Chess*. 262–276. doi:10.1007/3-540-45579-5_18

[18] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf

[19] Weihao Kong, Erez Louidor Ilan, Shuxin Nie, Taman Narayan, Rajat Sen, Yichen Zhou, Deqing Fu, Samet Oymak, and Abhimanyu Das. 2026. TabFM: A Zero-Shot Foundation Model for Tabular Data. arXiv:2609.37959 [cs.LG] https://arxiv.org/abs/2609.37959

[20] Scott Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. arXiv:1705.07874 [cs.AI] https://arxiv.org/abs/1705.07874

[21] Reid McIlroy-Young, Siddhartha Sen, Jon Kleinberg, and Ashton Anderson. 2020. Aligning Superhuman AI with Human Behavior: Chess as a Model System. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '20)*. ACM, 1677–1687. doi:10.1145/3394486.3403219

[22] Szymon Miłosz. 2025. Pretraining Transformers for Chess Puzzle Difficulty Prediction. In *2025 20th Conference on Computer Science and Intelligence Systems (FedCSIS)*. 831–835. doi:10.15439/2025F7603

[23] Daniel Monroe, George Eilender, Philip Chalmers, Zhenwei Tang, and Ashton Anderson. 2026. Chessformer: A Unified Architecture for Chess Modeling. arXiv:2605.19091 [cs.LG] https://arxiv.org/abs/2605.19091

[24] NVIDIA. 2026. Kumo-Tabular v1.0: Tabular Foundation Model. https://huggingface.co/nvidia/Kumo-Tabular. NVIDIA Structured Data Models.

[25] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[26] Danilo Machado PIRES and Júlio Sílvio de Sousa BUENO FILHO. 2020. Can ELO Ratings Be Improved? A Case Study with Elite Chess Players. *Brazilian Journal of Biometrics* 38, 4 (2020), 483–505.

[27] PyTuroChamp. [n. d.]. *PyTuroChamp*. https://github.com/stevexyz/PyTuroChamp

[28] Andry Rafaralahy. 2024. Pairwise Learning to Rank for Chess Puzzle Difficulty Prediction. In *2024 IEEE International Conference on Big Data (BigData)*. 8385–8389. doi:10.1109/BigData62323.2024.10825356

[29] Massimo Revelli. [n. d.]. *Classical Python Chess Engine*. https://github.com/massimorevelli/chess-engine

[30] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*. 1135–1144.

[31] Tarun Kumar S. 2026. Otter: A Time-Aware, History-Conditioned Human Chess AI. arXiv:2608.05206 [cs.AI] https://arxiv.org/abs/2608.05206

[32] Lars Sandin. 2022. The SSDF Chess Engine Rating List, 2022-01. *ICGA Journal* 43, 4 (2022), 236–239. arXiv:https://journals.sagepub.com/doi/pdf/10.3233/ICG-220204 doi:10.3233/ICG-220204

[33] Gesina Schwalbe and Bettina Finzel. 2023. A comprehensive taxonomy for explainable artificial intelligence: a systematic survey of surveys on methods and concepts. *Data Min. Knowl. Discov.* 38, 5 (Jan. 2023), 3043–3101. doi:10.1007/s10618-022-00867-8

[34] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. 2017. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. arXiv:1712.01815 [cs.AI] https://arxiv.org/abs/1712.01815

[35] Simon Stoiljkovikj, Ivan Bratko, and Matej Guid. 2015. A Computational Model for Estimating the Difficulty of Chess Problems. https://api.semanticscholar.org/CorpusID:204857616

[36] Sunfish. [n. d.]. *Sunfish*. https://github.com/thomasahle/sunfish

[37] Zhenwei Tang, Difan Jiao, Reid McIlroy-Young, Jon Kleinberg, Siddhartha Sen, and Ashton Anderson. 2024. Maia-2: A Unified Model for Human-AI Alignment in Chess. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=XWlkhRn14K

[38] The Stockfish developers. [n. d.]. *Stockfish*. https://stockfishchess.org/

[39] K. Thompson. 1982. Computer Chess Strength. In *Advances in Computer Chess*, M.R.B. CLARKE (Ed.). Pergamon, 55–56. doi:10.1016/B978-0-08-026898-9.50008-5

[40] Yiming Zhang, Athul Paul Jacob, Vivian Lai, Daniel Fried, and Daphne Ippolito. 2024. Human-aligned Chess with a Bit of Search. arXiv:2410.03893 [cs.LG] https://arxiv.org/abs/2410.03893

[41] Jan Zysko, Michael Slezak, Dominik Ślęzak, and Maciej Swiechowski. 2025. FedCSIS 2025 knowledgepit.ai Competition: Predicting Chess Puzzle Difficulty Part 2 & A Step Toward Uncertainty Contests. *2025 20th Conference on Computer Science and Intelligence Systems (FedCSIS)* (2025), 849–854. https://api.semanticscholar.org/CorpusID:282337586

[42] Jan Zyśko, Maciej Świechowski, Sebastian Stawicki, Katarzyna Jagieła, Andrzej Janusz, and Dominik Ślęzak. 2024. IEEE Big Data Cup 2024 Report: Predicting Chess Puzzle Difficulty at KnowledgePit.ai. In *2024 IEEE International Conference on Big Data (BigData)*. 8423–8429. doi:10.1109/BigData62323.2024.10825289

[43] Erik Štrumbelj and Igor Kononenko. 2014. Explaining prediction models and individual predictions with feature contributions. *Knowledge and Information Systems* 41 (2014), 647–665.